

A Unifying Semantics for Sequential Function Charts

Nanette Bauer¹, Ralf Huuck², Ben Lukoschus³, Sebastian Engell⁴

¹ BASF Aktiengesellschaft
67056 Ludwigshafen, Germany
`nanette.bauer@basf-ag.de`

² National ICT Australia Ltd (NICTA),
The University of New South Wales, Sydney, Australia
`rhuuck@cse.unsw.edu.au`

³ Institute of Computer Science and Applied Mathematics
University of Kiel, 24098 Kiel, Germany
`bls@informatik.uni-kiel.de`

⁴ Process Control Laboratory (BCI-AST)
University of Dortmund, 44221 Dortmund, Germany
`s.engell@bci.uni-dortmund.de`

Abstract. Programmable Logic Controllers (PLC) are widely used as device controllers for assembly lines, chemical processes, or power plants. Sequential Function Charts (SFC) form one of the main programming languages for PLCs and, therefore, the correctness of the PLC software implemented as SFCs is crucial for a safe operation of the controlled process. A prerequisite for reasoning about program correctness is a clear understanding of the program semantics. As we show in this work, this is currently not the case for SFCs. Although syntactically specified in the IEC 61131-3 standard, SFCs lack an unambiguous, complete semantic description. We point out a number of problems and explain how these lead to different interpretations in commercial programming environments. To remedy this situation we introduce a parameterized formal semantics for SFCs including many high-level programming features such as parallelism, hierarchy, actions and activity manipulation. Moreover, we show how to extend the semantics to include time, clocks, and timed actions. The presented semantics is general enough to comprise different existing interpretations while at the same time being adjustable to precisely represent each of them.

1 Introduction

Programmable Logic Controllers (PLCs) are widely used for automation in process industry, transportation and manufacturing. Their hard- and software is defined in the standard IEC 61131. Concerning the software different programming languages are standardized, one of them is Sequential Function Charts (SFC). SFCs are graphical high-level notations for PLC programs. They allow

the decomposition and structuring of program parts including interesting concepts like parallelism, activity manipulation and hierarchy. Moreover, there is the notion of time and timers, which allows to test and reason about the amount of time a program part has been active and to start program parts after a delay or for some limited time only. All these features can, however, again aggravate the understanding of SFCs.

As PLC software often performs safety-relevant functions, its formal analysis is crucial, and for this a formal semantics is required. However, the standard only gives an informal description of the semantics of the PLC languages. The semantics of SFCs given in IEC 61131-3 is ambiguous and erroneous, and PLC programming tool developers interpret the ambiguities posed by the standard in different ways, which results in a variety of different tool semantics. Although there exist a few approaches to formalize SFCs [1–3] none of them addresses all of the concepts which are crucial for SFCs, such as parallelism, hierarchy, their action concept, and timing aspects, and none of them considers tool-specific interpretations.

To remedy this situation a formal semantic is defined which includes all language concepts and furthermore tool-specific implementations. This semantics makes formal analysis possible and may also be helpful as a unifying basis for further tool developments.

The remainder of this work is organized as follows: First we introduce the basic concepts of SFCs informally in Sect. 2. In Sect. 3 we show ambiguities arising from the IEC 61131-3 definitions for SFCs and define a formal syntax and semantics for untimed SFCs which considers these ambiguities by the introduction of adaptable parameters. In Sect. 4 it is shown how this semantics can be augmented to timed aspects. Section 5 shows how the semantics can be adapted to the behavior of different tools. The contribution closes with conclusions given in Sect. 6.

2 Sequential Function Charts

2.1 Basic Structure

Sequential function charts are defined in [4] as elements of a graphical programming and structuring language for programmable logic controllers. The SFC definitions in IEC 61131-3 (in the following referred to as “the standard”) are based on IEC 60848 [5], which defines the specification language Grafset. Grafset in turn is strongly related to Petri nets [6].

Basically, SFCs are transition systems consisting of *steps* (the locations) and *transitions*. For every SFC there exists exactly one *initial step*. Every transition is labeled with an associated transition condition, called *guard*. Moreover, one or more *actions* may be associated to each step. Actions are again SFCs or programs in one of the other programming languages proposed by the standard. Since the actions associated to steps can also be SFCs themselves, a concept of hierarchy is provided. An example of a sequential function chart is depicted in Fig. 1.

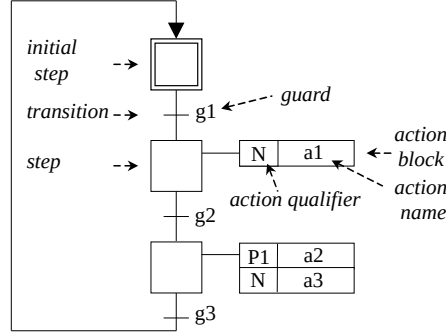


Fig. 1. Elements of SFCs

The *action blocks* shown in Fig. 1 are a graphical means to associate actions to steps. An action block consists of an *action qualifier*, which can be used to specify the activity of the respective action, and the *action name*. Concerning the action qualifiers defined in the standard, we focus on those without an associated duration of time (considered later in Sect. 4). These are the following:

qualifier	meaning
N	N on-stored
S	S et or S tored
R	R eset
P1	P ulse – rising edge (activation of corresponding step)
P0	P ulse – falling edge (deactivation of corresponding step)

Intuitively, the non-stored actions are always active while control resides in the corresponding step, i.e., as long as it is active. In contrast the stored actions remain active even outside their step of activation until the corresponding reset action is called. The actions with the pulse qualifier are performed only once when entering (P1) or exiting (P0) a step.

An SFC does not necessarily have to be a single sequence of steps and transitions. For SFCs we can identify a number of different transition types (cf. Fig. 2). Transition (1) denotes a *simple transition* between two steps, (2) describes *alternative branching*, i.e., the choice between several transitions, (3) *divergence*, (4) *convergence*, and (5) a simultaneity of both of the latter. In this context divergence means a parallel branching from one step to a set of next steps while convergence means the synchronization of several steps in parallel to a single one. Furthermore, direct combinations of both as depicted in (5) are allowed. Moreover, the standard refers to transitions like the one guarded by g_3 in Fig. 1 as *loops*. From our point of view loops as well as alternative branching belong to the same class, namely (sets of) simple transitions.

An additional feature of SFCs is to explicitly assign priorities to alternative branches. This means, when firing transitions the one which has the highest

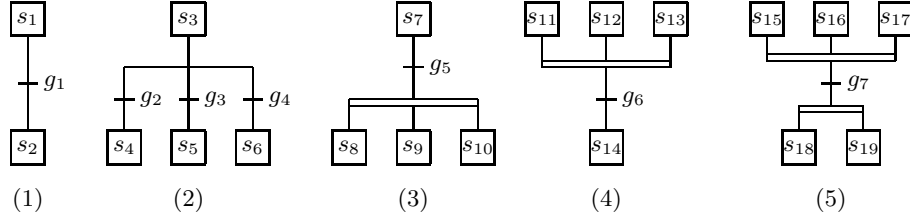


Fig. 2. Basic transition types

priority among the enabled ones will be taken. If there are no priorities given, SFC programming tools often implement the implicit rule that the transitions are ordered “from left to right” in decreasing priority, i.e., in Fig. 2 the transition guarded by g_2 has priority over those guarded by g_3 and g_4 , and the one guarded by g_3 has priority over the one guarded by g_4 .

The basic transition types can be combined into more complex transition structures like in Fig. 1 and Fig. 3. However, there are various combinations which do not seem to make sense, e.g., in the SFC shown in Fig. 3. There we have the transition from s_4 to s_1 which jumps out of a parallel branch, the converging transition from s_2 to s_5 , which jumps from one simultaneous branch to another, and the transition from s_5 and s_6 to s_7 , which is a convergence of simultaneous sequences of two steps which are part of an alternative branching starting in step s_3 .

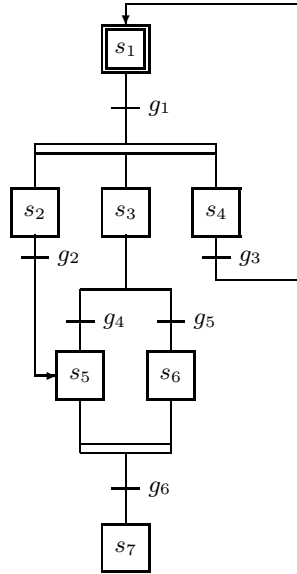


Fig. 3. Unsafe SFC

Although the standard forbids to use these kinds of “unsafe” or “unreachable” SFCs, it does not give a precise characterization of this phenomenon. For the time being we do not want to pose any explicit restrictions on the SFC syntax. Mainly because we want to allow as much freedom as possible and because there are commercial tools available which “support” SFCs with these kinds of transitions and do simulations on them. Hence, in the following we define a semantics for sequential function charts which can also cope with such ill-formed constructions.

2.2 Ambiguities

The standard defines rules for building an SFC from the aforementioned basic elements and describes how to execute SFCs by giving evolution rules similar to the firing rules of Petri nets. However, execution is only defined on an abstract level not taking into account concrete aspects of program execution. As SFC is a *programming language*, the exact semantics of execution is of interest and it is therefore important to consider the PLC-specific cyclic execution of SFCs. In every scan cycle first the current input from the environment (i.e., from the sensors of a plant such as pressure or temperature sensors) is read and stored. Then the PLC program is executed based on the stored input, i.e., the actions of the active steps are executed, which may change the output, and afterwards the transitions are taken. At the end of each cycle the output is sent to the environment, i.e., to the actuators of a plant such as valves or motors.

Although the semantics of SFCs on an abstract level seems to be quite simple, the exact semantics on an operational level is sometimes far from obvious. Let us have a close look at some key issues.

How to deal with parallelism? In which order are actions of parallel steps executed? Is there any order at all or do we have to cope with non-determinism? As illustrated by the example in Fig. 4 the order of execution makes a difference if the actions read or modify the same variables. Depending on whether executing a_1 before a_2 or not, the transition guarded with $y = 2$ is enabled or not.

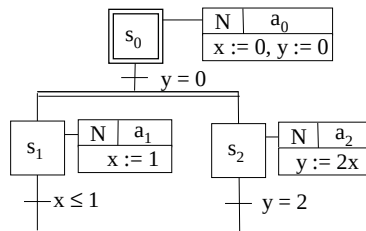


Fig. 4. In which order are the actions a_1 and a_2 executed?

The execution order is also not clear if we have more than one action associated to a step. Are they executed from top to bottom or according to a different rule?

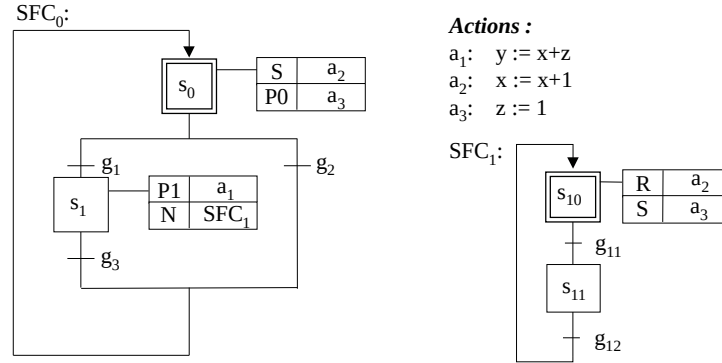


Fig. 5. How to deal with hierarchy?

How to deal with hierarchy of SFCs? Fig. 5 shows a top-level SFC SFC_0 with a second SFC nested in step s_1 (SFC_1). Has the top-level SFC higher priority, that is, are the actions of the top-level SFC executed prior to the actions of the nested SFC? If we enter s_1 in SFC_0 , does the execution order of actions depend on the hierarchy, i.e., do we first execute a_1 and then the actions a_2 and a_3 of SFC_1 ? Can SFC_1 reset a_2 or has the top SFC priority and a_2 is executed? Which step of the nested SFC will be entered if it is called again? For instance, if we enter step s_1 again, do we always activate s_{10} or is there a notion of history, and the last active step of SFC_1 (which may also be s_{11}) therefore gets activated?

2.3 Possible Answers to Ambiguities

The standard does not provide any explicit answer to the questions posed above. For the execution order of actions a comparison of different SFC programming tools of some major PLC vendors [7] yields that there exist various orders on actions, e.g., for one tool actions are executed according to the alphabetic order of action names, for another one by user-defined orders, and so on. However, in no case there is non-determinism. This seems to be reasonable since PLCs are in general deterministic. The semantics defined in the following copes with these different semantics by introducing a parameter for the order of actions. Another parameter is introduced for the order of transitions to reflect priorities if several transitions of an alternative branching are enabled simultaneously.

For the questions concerning history one may deduce from the standard that actions exist globally within the SFC, that means we have a global execution order within the SFC and its nested SFCs. This also means that a reset of an action will also effect the action if it is called within a nested SFC.

Furthermore we define that an SFC action may either have or have no history, such that it is either possible to always re-enter in the last active step or always re-enter in the initial step.

Finally, there is the question whether it is possible to create algebraic loops, e.g., is it possible that a nested SFC calls the top-level SFC again with a different qualifier? In general this is not forbidden, but in our framework we assume systems without algebraic loops and leave it to the program designer to avoid these.

These decisions are the basis for a clear non-ambiguous syntax and semantics of SFCs which is presented next.

3 Formal Untimed Syntax and Semantics

3.1 Syntax of SFCs with Actions

SFCs (or PLC programs in general) have different types of variables, such as input variables, output variables, and local variables. The values of the variables may belong to different data types such as Boolean or integer. The valid variable and data types are regulated by the standard IEC 61131-3, already mentioned above. We abstract from these different variable and data types by simply saying that a PLC has a set of variables X which may have different values. To describe the values of all variables we use the notion of a *state*:

Definition 1 (State). *A state is a type-preserving function σ assigning a value to each variable in X . Let Σ be the set of all states.*

A state (i.e., the values of the variables) can be modified by *state transformations*. In the standard different types of programming languages are defined for such state transformations. We abstract from these languages by simply saying that a state transformation is a function that transforms a given state into another state. Here state transformations are deterministic; in non-deterministic languages a state is transformed into one of several possible successor states.

Definition 2 (State transformation). *A state transformation is a function $f : \Sigma \rightarrow \Sigma$. Let F be the set of all state transformations.*

Each step of an SFC is labeled with a (possibly empty) set of *action blocks*, which are pairs of action names and qualifiers. We associate either a unique state transformation or a call of a nested SFC with an action name. Moreover, we simply say *actions* when in fact referring to action names.

Definition 3 (Action name, action block). *An action name is an identifier for either a state transformation $f \in F$ or an SFC S (defined below). An action*

block is a pair (a, q) consisting of an action name a and an action qualifier q , where $q \in \{N, S, R, P0, P1\}$.

Let B denote the set of all action blocks. For any action block $b \in B$ we denote by b_a the projection to the action name and by b_q the projection to the qualifier.

We define a *guard* as a Boolean expression reasoning about variables and step activities. Syntactically, the activity of step s_i is expressed by the term $s_i.X$.

Definition 4 (Guard). A guard g is a Boolean expression over variables and step activities. We denote the set of all guards by G .

Moreover, we need an order on actions as well as on transitions. These allow to determine in which order actions are executed and which transition will be taken if several are in conflict with each other. In conflict means that more than one alternative transition is enabled. Hence, the order on transitions rules out any non-determinism. Let *SFC* be the set of sequential function charts, which are defined next.

Definition 5 (SFC). A sequential function chart (*SFC*) is a 9-tuple $\mathcal{S} = (X, S, A, s_0, T, \text{block}, \sqsubseteq, \prec, h)$, where

- X is a finite set of variables,
- S is a finite set of steps,
- A is a finite set of actions, which might be *SFCs*,
- $s_0 \in S$ is the initial step,
- $T \subseteq (2^S \setminus \{\emptyset\}) \times G \times (2^S \setminus \{\emptyset\})$ is a finite set of transitions,
- $\text{block} : S \rightarrow 2^B$ is an action labeling function which assigns a set of action blocks to each step,
- $\sqsubseteq \subseteq A \times A$ is a total order on actions, used to define the order in which the active actions are to be executed,
- $\prec \subseteq T \times T$ is a partial order on transitions, to determine priorities on conflicting transitions, and
- $h \in \{0, 1\}$ is a parameter which determines whether the *SFC* is executed with history ($h = 1$) or without ($h = 0$).

We uniquely represent a transition by its set of source steps, its guard and its set of target steps.

When putting a sequential function chart into formal syntax the main task is to determine the orders $\sqsubseteq$ and $\prec$. As mentioned in Sect. 2.1 these orders are in general implementation dependent and should be chosen in a meaningful way or according to the respective programming tool.

For any *SFC* $\mathcal{S}$, we use $X_{\mathcal{S}}, S_{\mathcal{S}}, \dots, h_{\mathcal{S}}$ to denote the components of $\mathcal{S}$.

3.2 Semantics of SFCs with Actions

In this section we provide an operational semantics for *SFCs*. Let $\mathcal{S} = (X, S, A, s_0, T, \text{block}, \sqsubseteq, \prec, h)$ be an *SFC*, and let $\mathcal{S}_i = (X_i, S_i, A_i, s_{0,i}, T_i, \text{block}_i, \sqsubseteq_i, \prec_i,$

h_i), $i = 1, \dots, n$, be the SFCs nested recursively inside the steps of $\mathcal{S}$. For a global, flat access to the nested structure we distinguish between elements which are defined globally within the SFC and its nested SFC actions (the set of variables, the non-SFC actions, and the order on actions $\sqsubseteq$; for a flat access we have, e.g., $\bar{A} = A = A_1 = \dots = A_n$), and elements which are defined for each SFC differently (these are, e.g., the set of steps, the set of transitions, and the order on transitions). For a global access we define the global set of steps $\bar{S} = S \cup S_1 \cup \dots \cup S_n$; in the same way we extend the other components which differ for each SFC.

In the following we assume that the SFCs are nested in such a way that there are no algebraic loops, e.g., there are no conflicting circular action qualifiers for action names.

There are several things we have to keep track of when observing executions of the SFC $\mathcal{S}$. First, we need information about the current state of $\mathcal{S}$, i.e., the values of its variables. Moreover, we have to know in which steps of $\mathcal{S}$ and its sub-SFCs $\mathcal{S}_1, \dots, \mathcal{S}_n$ control resides and whose actions are to be performed in a cycle.

It is crucial to distinguish between *ready steps* and *active steps*. Active steps are the ones control resides in and their actions will be performed. On the other hand, there might be steps where control resides in, but their actions will not be performed. The reason is, these steps belong to a nested SFC $\mathcal{S}_i$ which is currently not activated, i.e., there is no action active which points to $\mathcal{S}_i$. Control is “waiting” there to resume. We call all steps where control resides in *ready steps*. Hence, each active step is also a ready step, but the converse does of course not hold.

Moreover, *active actions* are actions which will potentially be executed in the current SFC cycle. This means, unless there is no matching reset action these actions will be performed. *Stored actions* are the ones which have been tagged with an **S** qualifier and potentially keep on being active outside their step of activation.

We store the information about the state and the other information of $\mathcal{S}$ in a *configuration*:

Definition 6 (Configuration). A configuration c of $\mathcal{S}$ is a 5-tuple $(\sigma, \text{readyS}, \text{activeS}, \text{activeA}, \text{storedA})$, where $\sigma \in \Sigma$ is the state of the variables, $\text{readyS} \subseteq \bar{S}$ is the set of ready steps, $\text{activeS} \subseteq \bar{S}$ is the set of active steps, $\text{activeA} \subseteq \bar{A}$ is the set of active actions, and $\text{storedA} \subseteq \bar{A}$ is the set of stored actions, i.e., the ones which might remain active outside the step they were called. Moreover, let C denote the set of all configurations.

Such a configuration is modified in the *cycles* of a PLC. In a cycle the following sequence of tasks is performed:

1. Get new input from the environment and store the information into state σ .
2. Execute the set activeA of active actions and update σ accordingly.
3. Determine the set of next readyS , activeS , activeA , and storedA .
4. Send the outputs to the environment by extracting the required information from the new state σ .

We do not specify formally how to interact with the environment, but focus on items 2 and 3.

First, we define *executions* by means of configuration changes within a cycle. In order to do so, we associate a transition system to an SFC and use the following notations: For every transition (χ, g, χ') where χ and χ' are sets of steps let $source(\chi, g, \chi') = \chi$ denote the set of all source steps and $target(\chi, g, \chi') = \chi'$ the set of all target steps. We extend these notions to sets of transitions in a pointwise fashion.

A prerequisite for a program evaluation is an interpretation of the guards. We interpret a guard by the set of configurations it describes.

Definition 7 (Guard interpretation). *Given a guard $g \in G$ and a configuration $c \in C$, we say that “ c satisfies g ” or “ g is valid in c ”, denoted by $c \models g$, if the expression g evaluates to true under c .*

The semantics of an SFC is based on the transition system describing its operational behavior:

Definition 8 (Transition System of an SFC). *With every SFC $S = (X, S, A, s_0, T, block, \sqsubset, \prec, h)$ we associate a transition system $\mathcal{E}(S) = (C, c_0, \longrightarrow)$, where C is the set of configurations, $c_0 \in C$ is the initial configuration, and $\longrightarrow \subseteq C \times C$ is the transition relation, which is defined by: $(\sigma, readyS, activeS, activeA, storedA) \longrightarrow (\sigma', readyS', activeS', activeA', storedA')$ if and only if*

1. $\sigma' = (a_m \circ \dots \circ a_1)(\sigma)$, where $a_1 \sqsubset \dots \sqsubset a_m$ and $\{a_1, \dots, a_m\} = activeA \setminus SFC$,
2. $readyS' = (readyS \setminus source(taken)) \cup target(taken)$, where
 - (a) $enabled = \{(\chi, g, \chi') \in T \mid \chi \subseteq activeS \wedge (\sigma', readyS, activeS, activeA, storedA) \models g\}$, and
 - (b) $taken = \{t = (\chi, g, \chi') \in enabled \mid \neg \exists t_1 = (\chi_1, g_1, \chi'_1) \in enabled : \chi \cap \chi_1 \neq \emptyset \wedge t_1 \prec t\}$, and
3. $activeS'$, $activeA'$ and $storedA'$ are computed by

$$\begin{aligned}
 (\alpha_0, \beta_0) &:= \varphi(\alpha_{ini}, \emptyset, \mathcal{S}) \\
 (\alpha_1, \beta_1) &:= \varphi(\alpha_0, \beta_0, \mathcal{S}_{i_1}) \\
 &\vdots \\
 (\alpha_k, \beta_k) &:= \varphi(\alpha_{k-1}, \beta_{k-1}, \mathcal{S}_{i_k})
 \end{aligned}$$

where φ is defined in Fig. 6, $\mathcal{S}$ is the top-level SFC, the set $\{\mathcal{S}_{i_1}, \dots, \mathcal{S}_{i_k}\} = storedA \cap SFC$ contains the SFCs in $storedA$, and α_{ini} is the initial mapping of stored actions $a \in storedA \setminus SFC$ to $\{\mathcal{S}\}$ and to $\emptyset$ for all other actions. Then, $activeS' = \beta_k$, $activeA' = \{a \in \bar{A} \mid \alpha_k(a) \cap \{\mathbf{N}, \mathbf{P0}, \mathbf{P1}\} \neq \emptyset \wedge \mathbf{R} \notin \alpha_k(a)\}$ and $storedA' = \{a \in \bar{A} \mid \mathcal{S} \in \alpha_k(a) \wedge \mathbf{R} \notin \alpha_k(a)\}$.

The initial configuration $c_0 = (\sigma_0, readyS_0, activeS_0, activeA_0, storedA_0)$ is given by: σ_0 is the predefined initial state of the variables (Booleans set to false, numerical variables to 0), $readyS_0 = \bar{s}_0$, and $activeS_0$, $activeA_0$, and $storedA_0$ are determined from (α_0, β_0) like the primed ones above.

```

function  $\varphi(\alpha, \beta, \Gamma)$ 
  (* (I) newsteps are the local ready steps, i.e., the active ones *)
  if  $h_\Gamma = 1$ 
    then  $\text{newsteps} := S_\Gamma \cap \text{ready}S'$ 
    else  $\text{newsteps} := S_\Gamma \cap \bar{s}_0$ ;
   $\alpha' := \alpha$ ;
   $\beta' := \beta \cup \text{newsteps}$ ;
  (* (II) collect the relevant actions *)
  for all  $s \in S_\Gamma, b \in \text{block}(s)$ :
     $\alpha'(b_a) := \alpha'(b_a) \cup \begin{cases} \{P0\} & \text{if } b_q = P0 \wedge s \in \text{source}(\text{taken}) \\ \{P1\} & \text{if } b_q = P1 \wedge s \in \text{target}(\text{taken}) \\ \{b_q\} & \text{if } b_q \in \{N, S, R\} \wedge s \in \text{newsteps} \end{cases}$ 
  (* (III) go into recursion for every new SFC which is not reset *)
  for all  $s \in \text{newsteps}, b \in \text{block}(s)$ :
    if  $b_a \in \text{SFC} \wedge R \notin \alpha'(b_a) \wedge \alpha'(b_a) \neq \emptyset$ 
      then  $(\alpha', \beta') := \varphi(\alpha', \beta', b_a)$ ;
  return  $(\alpha', \beta')$ 

```

Fig. 6. Recursive action labeling collection

Let us take a detailed look at the definition above. One transition in the transition system above corresponds to one PLC cycle. First of all in step 1 of Definition 8 the current active actions are ordered and executed accordingly. If two active actions access a common variable (e.g., x in Fig. 4), the result may depend on the action ordering $\sqsubset$.

In step 2 we determine the new ready steps. These are the old ones plus the targets of the taken transitions, but without their source steps. We distinguish *taken* and *enabled* transitions as follows: A transition is enabled if it is an outgoing transition of an active step and its guard is satisfied by the current configuration. A transition t is taken if it additionally has the highest priority among its competing ones (transitions t_1 sharing at least one common source step with t), which is expressed by the $\bar{\succ}$ relation.

In step 3 the new active steps, active actions and stored actions are computed recursively on the structure of the SFC by the auxiliary function φ . The function φ has the parameters α , β , and Γ , where α is a mapping from action names to sets of qualifiers and $\beta \subseteq \bar{S}$ is a set of steps. Calling φ for a specific SFC Γ with empty α and β results in the pair (α', β') , where α' contains for every active action all the “activated” qualifiers of the active actions in Γ and its recursively nested SFCs, and β' contains all active steps.

This is computed as follows: starting on with the top-level of Γ , we compute in (I) *newsteps*, which is the set of active steps on this level. The function α is copied to α' and β' is the old β conjoined with the new active steps. In (II), α' is extended by further qualifiers for specific actions. The inclusion of qualifiers depends on whether these qualifiers become relevant now or not. The pulse qualifiers are relevant when taking a transition. The other qualifiers are relevant when remaining in the active steps (i.e., *newsteps*). Finally, in (III) we go into

recursion for every sub-SFC of the current one which has become active after computing the new active steps and implicitly the new active actions. These are the SFCs associated to the new steps which are not reset.

The recursion of φ is finite since we assumed that the nested SFC structure does not contain algebraic loops, see Sect. 2.2.

In step 3 of Definition 8 φ is called for the initial set of α , which is just the mapping of stored actions to the corresponding qualifier and the empty set for β . φ then needs to be called again for all k stored SFCs which are in $storedA$. The result of the application of φ is (α_k, β_k) , where β_k corresponds to the new active steps, and $activeA'$ and $storedA'$ are the projections of α_k to the corresponding qualifiers taking into account any resets.

An execution sequence of an associated transition system is called *run*.

Definition 9 (Run of $\mathcal{E}$). A run r over a transition system $\mathcal{E} = (C, c_0, \longrightarrow)$ is a finite or infinite sequence $\langle c_0, c_1, \dots \rangle$ with $c_i \longrightarrow c_{i+1}$ for all i .

The operational semantics $\llbracket \mathcal{S} \rrbracket$ for an SFCs $\mathcal{S}$ is given by the set of runs of its associated transitions system $\mathcal{E}(\mathcal{S})$.

3.3 Example

For the SFC given in Fig. 5 we explain how the semantics works for crucial issues such as actions, where the execution order matters, and hierarchy of SFCs.

Assume the global order on actions is given by $a_1 \sqsubset a_2 \sqsubset a_3$, and the local order on transitions is given by $g_1 \prec g_2$. Let the initial configuration for the SFC be (we write functions as sets of pairs):

$$\begin{aligned}\sigma_0 &= \{(x, 0), (y, 0), (z, 0)\}, \\ readyS_0 &= \bar{s}_0 = \{s_0, s_{10}\}, \\ activeS_0 &= \{s_0\}, \\ activeA_0 &= \{a_2\}, \text{ and} \\ storedA_0 &= \{a_2\}.\end{aligned}$$

Furthermore we assume that the action SFC_1 has history, i.e., $h_1 = 1$. Top-level SFCs always have history ($h = 1$).

Assume, g_1 and g_2 are not true until we are in cycle number 14, but then both guards g_1 and g_2 evaluate to true. At this point

$$\sigma = \{(x, 13), (y, 0), (z, 0)\},$$

the sets of active steps, ready steps etc. have not changed compared to the initial configuration. We compute according to step 1 of Definition 8:

$$\sigma' = a_2(\sigma) = \{(x, 14), (y, 0), (z, 0)\},$$

and according to step 2 of the same definition:

$$readyS' = (\{s_0, s_{10}\} \setminus \{s_0\}) \cup \{s_1\} = \{s_1, s_{10}\}.$$

Moreover, $enabled = \{(\{s_0\}, g_1, \{s_1\}), (\{s_0\}, g_2, \{s_0\})\}$, and since $(\{s_0\}, g_1, \{s_1\}) \prec (\{s_0\}, g_2, \{s_0\})$, we have $taken = \{(\{s_0\}, g_1, \{s_1\})\}$. For the computation of the new active steps and actions we have to take the nested SFC_1 into account and recursively determine $activeS'$, $activeA'$ and $storedA'$ using the auxiliary function φ . Because $storedA \cap SFC = \emptyset$, i.e., there is no stored SFC, we only need to compute $\varphi(\alpha_{ini}, \emptyset, SFC_0)$ for the top-level SFC which is SFC_0 . The initial mapping α_{ini} of stored actions is given by

$$\begin{aligned}\alpha_{ini}(a_1) &= \emptyset, \\ \alpha_{ini}(a_2) &= \{S\}, \\ \alpha_{ini}(a_3) &= \emptyset, \text{ and} \\ \alpha_{ini}(SFC_1) &= \emptyset.\end{aligned}$$

First, we determine the active steps of the top-level SFC by

$$newsteps = \{s_0, s_1\} \cap \{s_1, s_{10}\} = \{s_1\}.$$

We set $\alpha' = \alpha_{ini}$ and $\beta' = \{s_1\}$. To update α' , all relevant action qualifiers are collected. This yields

- $\alpha'(a_3) = \{P0\}$, since $(a_3, P0)_q = P0$ and $s_0 \in source(taken)$,
- $\alpha'(a_1) = \{P1\}$, since $(a_1, P1)_q = P1$ and $s_1 \in target(taken)$, and
- $\alpha'(SFC_1) = \{N\}$, since $(SFC_1, N)_q \in \{N, S, R\}$ and $s_1 \in newsteps$.

Next, we go into recursion because there is a new SFC, which is not reset, i.e., $SFC_1 \in SFC$, $R \notin \alpha'(SFC_1)$ and $\alpha'(SFC_1) \neq \emptyset$. Therefore, we compute $(\alpha', \beta') := \varphi(\alpha', \beta', SFC_1)$. This results in

$$newsteps = S_1 \cap readyS' = \{s_{10}\}, \alpha' = \alpha, \text{ and } \beta' = \{s_1, s_{10}\}.$$

The update of α' by collecting the relevant actions for all steps of SFC_1 yields

- $\alpha'(a_2) = \{S, R\}$, since $(a_2, R)_q \in \{N, S, R\}$ and $s_{10} \in newsteps$ and
- $\alpha'(a_3) = \{P0, S\}$, since $(a_3, S)_q \in \{N, S, R\}$ and $s_{10} \in newsteps$.

Since there are no new SFCs, the function φ terminates and returns $\beta_0 = \beta'$ as given above and $\alpha_0 = \alpha'$ with

$$\begin{aligned}\alpha_0(a_1) &= \{P1\}, \\ \alpha_0(a_2) &= \{S, R\}, \\ \alpha_0(a_3) &= \{P0, S\}, \text{ and} \\ \alpha_0(SFC_1) &= \{N\}.\end{aligned}$$

Hence, we obtain

$$activeS' = \beta_0 = \{s_1, s_{10}\}, activeA' = \{a_1, a_3, SFC_1\}, \text{ and } storedA' = \{a_3\}.$$

Note, if we continue to compute the configuration for the next cycle 15, the order on actions is important as more than one action is active:

$$\sigma' = (a_3 \circ a_1)(\sigma) = \{(x, 14), (y, 14), (z, 1)\}.$$

In this case a different order on a_1 and a_3 would lead to a different configuration where $\sigma'(y) = 15$. Furthermore, since we do not reset a_3 which is set in step s_{10} of SFC_1 , the action a_3 will still be executed even if SFC_1 is no longer active.

4 Formal Extension to Timed SFCs

The SFCs presented earlier do not support any time or timing behavior. However, the standard defines the notion of time and timers which allows to test and reason about the amount of time a step has been active and to start actions after a delay or for some limited time only. In this section we give an introduction to timed SFCs. We explain the main features and how to extend the current SFC semantics to timed SFCs.

4.1 Timed Syntax

Guards are Boolean expressions over variables, where $s_i.X$ denotes that step s_i is active and $s_i.T$ is the time that s_i has been active since its last activation. In addition to the qualifiers presented in Sect. 2.1 there are a number of time related qualifiers:

qualifier	meaning
L t	L imited execution for time span t
D t	D elayed execution after time span t
SL t	S et L imited execution for time span t
SD t	S et D elayed execution after time span t
DS t	D elayed S et execution after time span t

All these qualifiers are followed by some duration and their meaning is as follows: The limited qualifier **L** behaves like the non-stored qualifier **N** with the only difference that the corresponding action becomes at latest inactive when the associated duration has elapsed. Moreover, it is possible to delay the activation of an action for a certain time after the step has become active using the **D** qualifier. The **L** and **D** qualifiers can be combined with **S** yielding **DS**, **SD** and **SL**. An action associated with **DS** will only be activated if the delay time is reached *before* the step is left, whereas an **SD** action always becomes active after the elapsed time, independent of the step activity. Similar, an **SL** action is (in contrast to **L**) *always* activated for t time units. We denote the action labeling function extended by the additional qualifiers by $block^*$.

Although the action qualifier concept is defined using well-understood function blocks, the definitions in the standard still are ambiguous. For example, assume an SFC which calls an **SD** action in one step and in the next step calls the action again with the **SD** qualifier but with a different delay time. Assume that the activity time of the first step has been shorter than the first delay time, i.e., the action is not yet activated when it is called in the next step with a different delay time. Which delay time now is relevant? These ambiguities clearly show that there is a need for an SFC semantics which gives answers to these open questions. Here, we assume that any call of a timed action resets the corresponding timer. This means delays or limited actions restart the moment they are called. However, other interpretations are possible.

To reason about timed SFCs we introduce the notions of *clocks* and *stopwatches*.

Definition 10 (Stopwatch, Clock). A stopwatch θ_s is a real valued variable that has a dynamical behavior given by either $f(\theta_s) = \dot{\theta}_s = 1$ if the stopwatch is running, or $f(\theta_s) = \dot{\theta}_s = 0$ if it is stopped, whereas a clock θ_a cannot be stopped, i.e., the dynamical behavior is always $f(\theta_a) = \dot{\theta}_a = 1$. Both can be reset to zero.

To describe the values of all stopwatches and clocks we use the notion of a *clock evaluation* ν , which is a function assigning a value to each stopwatch or clock.

Since guards may reason about the time a step is active or has been active before, each step needs a stopwatch which is reset to zero when the step is entered, runs when the step is active, and is stopped when the step is deactivated. Second, we need stopwatches because we define that hierarchically nested SFCs have history, i.e., after deactivation nested SFCs are activated in the step they have been last. The notion of history includes that the SFCs remember the values of their clocks at the point of deactivation if they are activated again.

Additionally we need a clock for those actions which are associated with an SD or SL qualifier, because these actions might be deactivated (SL) or activated (SD) independently from a step activity. Clocks are sufficient, because according to the standard, guards cannot access the time an action has been active, and, therefore, we do not need to memorize the activity time after the action has been deactivated.

We define a timed SFC as follows:

Definition 11 (Timed SFC). A sequential function chart (SFC) $\mathcal{S} = (X, \Theta, S, A, s_0, T, block^*, \sqsubset, \prec, h)$ consists of:

- $X, S, A, s_0, T, \sqsubset, \prec$ and h as defined for the untimed SFC (Definition 5),
- $block^*$, the extended action labeling function, and
- a finite set Θ of stopwatches and clocks.

4.2 Timed Semantics

The global state of an SFC (including all its nested SFCs) is given by the values of all its variables, the evaluation of clocks and stopwatches, the sets of active and ready steps and the sets of active and stored actions. In addition to stored actions we also have to remember *stored delayed actions* associated to a step with an SD qualifier, which potentially need to be activated after the corresponding step has been deactivated, as well as *stored limited actions* indicated by the SL qualifier, which potentially have to be executed for a certain time after the activating step has been deactivated.

We describe the global state of an SFC $\mathcal{S}$ by a configuration, given by:

Definition 12 (Timed Configuration). A configuration c of $\mathcal{S}$ is an 8-tuple $(\sigma, \nu, readyS, activeS, activeA, storedA, storedDA, storedLA)$, where

- $\sigma, \text{readyS}, \text{activeS}, \text{activeA}, \text{storedA}$ are defined as for the untimed configuration (Definition 6) and
- ν is the evaluation of clocks and stopwatches,
- $\text{storedDA} \subseteq \bar{A}$ is the set of stored delayed actions, and
- $\text{storedLA} \subseteq \bar{A}$ is the set of stored limited actions.

The timed SFC program executions are again defined by means of configuration changes within a cycle. We define two different types of transitions, $c \xrightarrow{a}_t c'$ which describes the changes of the configuration due to the program execution (which is considered to take no time) and $c \xrightarrow{\delta}_t c'$ which describes the change of the configuration due to the elapse of time, this means it describes the change of ν .

A transition $c \xrightarrow{a}_t c'$ in this transition system is computed by the following sequence:

1. Determine the new state σ' and readyS' as for the untimed configuration.
2. Determine $\text{activeS}', \text{activeA}', \text{storedA}', \text{storedDA}'$ and $\text{storedLA}'$. The new sets are computed recursively on the structure of the SFC using the auxiliary function φ as defined in Sect. 3.2 but augmented with the handling of the timed action qualifiers.
3. Stopwatches and clocks are reset to zero if one of the following conditions holds:
 - The stopwatch θ_s belongs to a step s which has been activated in this cycle, i.e., $s \in \text{activeS}' \setminus \text{activeS}$.
 - The clock θ_a belongs to an action a which has been activated “stored delayed” in this cycle, i.e., $a \in \text{storedDA}' \setminus \text{storedDA}$.
 - The clock θ_a belongs to an action a which has been activated “stored limited” in this cycle, i.e., $a \in \text{storedLA}' \setminus \text{storedLA}$.

A transition $c \xrightarrow{\delta}_t c'$ in this transition system changes ν as follows:

- $\nu'(\theta_s) = \nu(\theta_s) + \delta$ if and only if the stopwatch θ_s belongs to a step $s \in \text{activeS}$, i.e., a step which is active, and
- $\nu'(\theta_a) = \nu(\theta_a) + \delta$ for all clocks θ_a .

A timed execution sequence $\langle c_0 \xrightarrow{\delta_0}_t c_1 \xrightarrow{a}_t c'_1, \dots \xrightarrow{\delta_i}_t c_{i+1} \xrightarrow{a}_t c'_{i+1} \dots \rangle$ is called *timed run*. Note that the time which elapses between two program executions is δ_i , which is the cycle time of the PLC. In PLCs, the cycle time usually is not fixed; some PLCs allow to define a fixed cycle time or an upper time bound. The operational semantics of an SFC $\mathcal{S}$ is given by the set of runs of its associated transition system.

5 Adaption to Tool Semantics

As already mentioned, the crucial step for adapting the semantics to a specific tool semantics is the definition of the orders on actions $\sqsubset$ and the order on transitions $\prec$ as implemented in the tool.

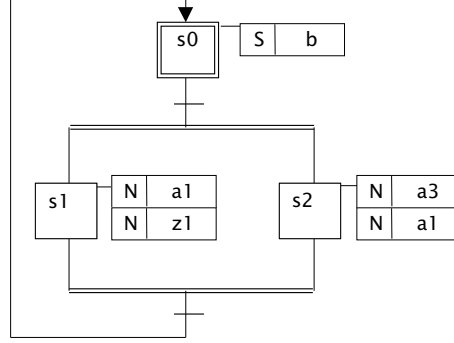


Fig. 7. Example for the ordering $\sqsubset$ on actions

For the example presented in Fig. 7 we show how the order can be adapted to different orders on actions which we found to be implemented in different SFC programming tools of some major PLC vendors, for details see [8]:

1. Alphabetical order:

$$a_1 \sqsubset a_3 \sqsubset b \sqsubset z_1.$$

2. Graphical order, from left to right and top to bottom:

$$b \sqsubset a_1 \sqsubset z_1 \sqsubset a_3 \sqsubset a'_1.$$

As implemented in the tool, the action a_1 is executed twice, which is not in accordance with the standard. It has to be considered also twice in the order on actions, which can be achieved by introducing two different action names a_1 and a'_1 .

3. The order is defined by the user, for example:

$$z_1 \sqsubset a_1 \sqsubset a_3 \sqsubset b.$$

The adaption works similarly for the different transition orders which were found to be implemented in programming tools. Consider the examples presented in Fig. 8. The corresponding orders on transitions are (for the sake of readability, we use t_i as a placeholder for the transition guarded by g_i when describing the order $\prec$):

1. No explicit or implicit priorities, SFC (a) in Fig. 8:

$$\prec = \emptyset.$$

If there are no priorities at all, it is recommended to ensure that conflicting transitions are never enabled simultaneously, e.g., by selecting disjoint guards.

2. Priority from left to right, denoted by a *-notation (this is defined in the standard), SFC (b) in Fig. 8:

$$\prec = \{(t_1, t_{12}), (t_1, t_3), (t_{12}, t_3)\}.$$

3. Priorities explicitly given by the user, SFC (c) in Fig. 8

$$\prec = \{(t_3, t_1), (t_3, t_{12}), (t_1, t_{12})\}.$$

4. The leftmost transition always has priority if it is enabled. If it is not enabled, the priority is defined by a transition number which is automatically generated by the programming tool. For this example, assume the numbers to be as given for SFC (c) in Fig. 8 :

$$\prec = \{(t_1, t_{12}), (t_1, t_3), (t_3, t_{12})\}.$$

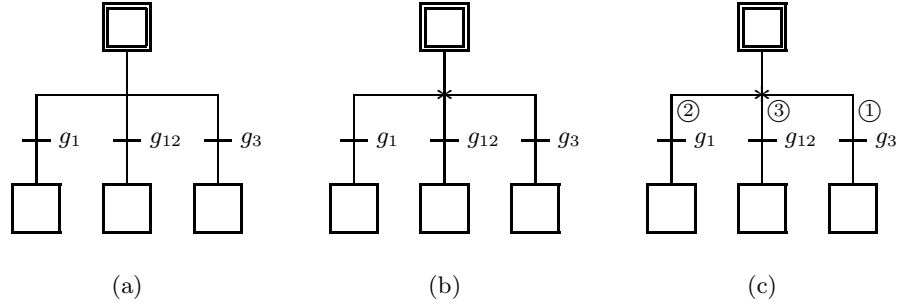


Fig. 8. Example for the ordering $\prec$ on transitions

6 Conclusions

In this work we presented a number of ambiguities and incomplete definitions of the IEC 61131-3 standard for the PLC programming and structuring language Sequential Function Charts (SFC). Moreover, we pointed out that these shortcomings can lead to different implementations of the same SFC, which poses a potential safety hazard. A formal semantics is a first step to avoid such threats. We proposed a formal semantics for untimed SFCs and an extension of this semantics to timed SFCs. This semantics covers the particularities of SFCs such as parallelism, hierarchy, actions and activity manipulations. Moreover, the introduced parameters on actions and transitions gives the semantics the flexibility to adjust precisely to a number of existing interpretations found in PLC programming tools.

Future work is to develop a simple and “clean” semantics and propose it for a future edition of the standard to support the portability of SFC programs among users of different PLC development tools.

Acknowledgments

This work was supported by the German Research Council within the DFG Priority Programme “Integration of Software Specification Techniques for Applications in Engineering” under grants RO 1122/10-2 and EN 152/32-2.

National ICT Australia is funded through the Australian Government's *Backing Australia's Ability initiative*, in part through the Australian Research Council. Nanette Bauer contributed to this work during her time at the Process Control Group of Prof. Engell, University of Dortmund.

Finally, we thank the anonymous referees for their comments.

References

1. Jiang, J., Holding, D.: The formalisation and analysis of sequential function charts using a petri net approach. In: Proceedings of 13th World Congress of IFAC. (1996) 513–518
2. Anderson, S., Tourlas, K.: Design for proof: An approach to the design of domain-specific languages. *Formal Aspects of Computing* **10** (1998) 452–468
3. Bornot, S., Huuck, R., Lakhnech, Y., Lukoschus, B.: An abstract model for sequential function charts. In: Discrete Event Systems: Analysis and Control, Proceedings of WODES 2000: 5th Workshop on Discrete Event Systems, Ghent, Belgium, August 21–23, 2000. The Kluwer International Series in Engineering and Computer Science (2000) 255–264
4. International Electrotechnical Commission, Technical Committee No. 65: Programmable Controllers – Programming Languages, IEC 61131-3. second edn. (1998) Committee draft.
5. International Electrotechnical Commission, Technical Committee No. 848: IEC 60848, Preparation of function charts for control systems. (1992)
6. David, R., Alla, H.: Petri Nets & Grafcet. Prentice Hall (1992)
7. Bauer, N., Treseler, H.: Vergleich der Semantik der Ablaufsprache nach IEC 61131-3 in unterschiedlichen Programmierwerkzeugen. GMA Kongress, 2001, Baden-Baden, Germany (2001)
8. Bauer, N.: Formale Analyse von Sequential Function Charts. PhD thesis, Lehrstuhl für Anlagensteuerungstechnik, Universität Dortmund (2004) Shaker Verlag, Aachen.